Low-Code Experiment

Applying Performance Optimization Techniques (Indexing, Subforms, Query Tuning) to a Zoho Creator Application

Sachitha S
241501171
AIML

Abstract
This experiment looks at performance optimization in the HOTEL BOOKING application, with the focus on query tuning in Deluge. Custom Deluge functions were created in the Workflow area to count Suite bookings, and the untuned loop-based approach is compared with the tuned approach that lets the database filter and count the records. Indexing and subforms are explained as the other optimization techniques.

1. Aim
To apply performance optimization techniques such as indexing, subforms and query tuning to a Zoho Creator application.

2. Concepts
Indexing and typed fields. Fields that are used repeatedly in report filters and fetch criteria, such as Room Type and Check-in date, should be kept as typed fields (Drop Down, Date) rather than free text so that criteria on them are fast. Zoho Creator manages the storage index itself.

Subforms. A subform stores several related rows (for example each additional guest) inside one parent record instead of separate form submissions, which reduces the number of records and queries.

Query tuning. In Deluge, filter with criteria in the fetch, use count and aggregate tasks instead of loops, fetch with a range, and never fetch inside a loop. In reports, apply criteria and show only the needed columns.

Functions. The Functions tab of the Workflow area holds reusable Deluge functions with a name, language, return type and syntax. They can be called from workflows, which keeps tuned logic in one place.

3. Procedure
1. Open the Workflow area and click the Functions tab, next to Form workflows, Schedules, Approvals, Payments, Blueprints and Batch workflows.
2. Click New Function and create a Deluge function in the Default namespace for counting Suite bookings with the untuned loop method shown in the code section.
3. Create a second function with the tuned method, which counts with criteria in a single query.
4. Check the Functions list, which shows each function with its language (Deluge), return type (void) and syntax.
5. Compare the two approaches: the untuned one fetches every record and loops, and the tuned one asks the database for the count.

4. Deluge / Configuration Code
```
// Untuned: fetches every record, then counts in a loop
```

```
all = Room_Booking_Form[ID != 0];
total = 0;
for each r in all
{
    if(r.Room_Type == "Suite")
    {
        total = total + 1;
    }
}

// Tuned: let the database filter and count
total = Room_Booking_Form[Room_Type == "Suite"].count();

// Tuned fetch: criteria plus a limited range
page1 = Room_Booking_Form[Room_Type == "Suite"] range from 1 to 50;

// Tuned: avoid fetch inside a loop; fetch once, reuse the collection
```

5. Implementation Screenshots (HOTEL BOOKING application)

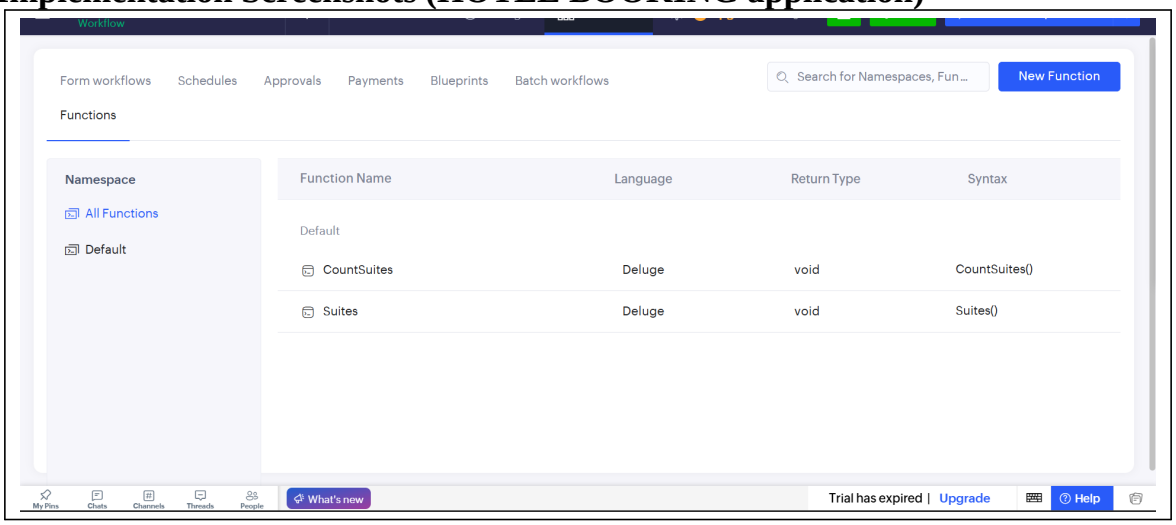


Figure 1: Functions tab of the Workflow area showing the Deluge functions CountSuites() and Suites() in the Default namespace

6. Expected Output

Both approaches give the same Suite count, but the tuned one uses a single database query with criteria instead of fetching all records and looping through them, so it stays fast as the number of bookings grows.

7. Result

The procedure above was prepared for the HOTEL BOOKING application. The observed results are recorded through the screenshots in the figures above. Where a feature is restricted because the free trial has expired, the message shown by Zoho Creator is included as the screenshot and treated as the observed result.

8. Learning Outcome

Learnt how field types, report criteria, subforms and query tuning improve the speed and scalability of a Creator application, how reusable Deluge functions are created in the Functions tab, and why filtering and counting should be done by the database rather than in Deluge loops.